



1. (20 نمره)

الف- (10 نمره) آرایه ای از اعداد صحیح به طول n داریم. شبه کدی بنویسید که از کاربر عدد صحیح k را دریافت کند و بررسی کند که آیا در آرایه، دو عدد وجود دارند که ضرب آنها برابر k شود یا خیر.

مثال) $\text{Array} = [6, 1, 3, 11, 19, 7]$

$k = 6 \rightarrow \text{return TRUE}$

$k = 12 \rightarrow \text{return FALSE}$

$k = 21 \rightarrow \text{return TRUE}$

پاسخ: تمام ترکیب های دوتایی داخل آرایه را در هم ضرب می کنیم و بررسی می کنیم که آیا حاصل برابر k می شود یا خیر.

```
read k, n, Array[n]
for i from 0 to n - 1 do
  for j from i + 1 to n - 1 do
    if Array[i] * Array[j] == k then
      return TRUE
    endif
  endfor
endfor
return FALSE
```

ب- (10 نمره) تابعی در زبان C به نام `checkMult` بنویسید که آرایه، طول آن (n) و عدد k را به عنوان آرگومان دریافت کند و الگوریتم بند الف را اجرا کند. مقدار برگشتی این تابع، 1 برای حالت TRUE (زمانی که دو عدد در آرایه وجود دارند که ضربشان برابر k است) و 0 برای حالت FALSE (دو عددی که ضربشان برابر k شود در آرایه وجود ندارد)، خواهد بود.

```
int checkMult(int Array[], int n, int k){
```

}

پاسخ:

```
int checkMult(int Array[], int n, int k) {  
    for(int i = 0 ; i < n ; i++){  
        for(int j = i + 1 ; j < n ; j++){  
            if (Array[i] * Array[j] == k){  
                return 1;  
            }  
        }  
    }  
    return 0;  
}
```

2. (20 نمره) فرض کنید یک تصویر سیاه و سفید داریم که به صورت یک آرایه 10×10 از اعداد صحیح (int) ذخیره شده است. هر خانه از این جدول، نشان‌دهنده‌ی روشنایی آن پیکسل است (مثلاً عددی بین 0 تا 255).

تابعی بنویسید که آرایه‌ی دو بعدی تصویر و مختصات یک پیکسل یعنی (i, j) را دریافت کند و مجموع روشنایی آن پیکسل و تمام همسایه‌های مجاورش را برگرداند.

```
int calculateBrightness(int image[10][10], int i, int j){  
}
```

نکته: منظور از همسایه‌ها، حداکثر ۸ خانه‌ی اطراف (بالا، پایین، چپ، راست و مورب‌ها هر کدام در صورت وجود) است. یعنی باید مجموع مقادیر یک مربع 3×3 که (i, j) در مرکز آن است محاسبه شود. برای مثال در شکل زیر، تمام خانه‌هایی که با X مشخص شده‌اند همسایه‌ی خانه A هستند:

X	X	X
X	A	X
X	X	X

دقت کنید که اگر خانه در لبه‌های تصویر باشد، فقط همسایه‌های موجود باید در نظر گرفته شوند.

```

#define SIZE 10
int calculateBrightness(int image[SIZE][SIZE], int i, int j) {

    int sum = 0;
    for(int r = i - 1; r <= i + 1; r++){

        for(int c = j - 1; c <= j + 1; c++){
            // Check if r,c are in range 0 to 9
            if(r >= 0 && r < SIZE && c >= 0 && c < SIZE){
                sum += image[r][c];
            }
        }
    }

    return sum;
}

```

3. (20 نمره) یک آرایه‌ی تک‌بعدی از اعداد صحیح با طول n در نظر بگیرید. تابعی به نام `shiftRight` بنویسید که آرایه `(arr)`، طول آن (n) و تعداد واحدهای شیفت (k) که $k < n$ را به عنوان ورودی دریافت کند. این تابع باید آرایه را به تعداد k واحد به صورت دورانی به سمت راست شیفت دهد؛ به این معنی که عناصر خارج شده از سمت راست آرایه، باید از سمت چپ آن دوباره وارد شوند. (راهنمایی: می‌توانید از یک آرایه کمکی استفاده کنید)

```

void shiftRight(int arr[], int n, int k){

}

```

```

void shiftRight(int arr[], int n, int k) {

    int temp[n];
    // 1. Copy the last k elements (from N-k to N-1) to the temporary array.
    for(int i = 0; i < k; i++){
        temp[i] = arr[n - k + i];
    }

    // 2. Shift the remaining(N - k) elements to the right by k positions.
    for(int i = n - 1; i >= k; i--){
        arr[i] = arr[i - k];
    }

    // 3. Copy the k elements from the temp array to the start of the main array.
    for(int i = 0; i < k; i++){
        arr[i] = temp[i];
    }
}

```

4. (15 نمره) شبه کد الگوریتمی را بنویسید که آرایه‌ای به طول n از اعداد صحیح را دریافت کرده و میانگین بزرگترین عدد و کوچکترین عدد داخل آرایه را در خروجی چاپ کند. (استفاده از بیش از 1 حلقه مجاز نیست)

ابتدا دو متغیر با نام های max و min تعریف میکنیم و سپس هر دو را با اولین عدد آرایه مقاردهی اولیه میکنیم. سپس با پیمایش روی آرایه، اعداد را تک تک با متغیر های max و min مقایسه میکنیم و در صورت لزوم، مقادیر max و min را به روز میکنیم.

```
read Array[n]
initialize both max and min to Array[0]
for i from 1 to n - 1 do
    if Array[i] > max then
        max = Array[i]
    else if Array[i] < min then
        min = Array[i]
    endif
endfor
print (min + max) Div 2
```

5. **امتیازی (15 نمره) تابع زبان C زیر را طوری کامل کنید که یک رشته حرفی شامل حروف انگلیسی و علائم نگارشی با طول دلخواه را از کاربر دریافت کند و رشته حرفی را اینگونه تغییر دهد:

- 1) رشته حرفی را برعکس کند (از آخر به اول).
- 2) حروف انگلیسی بزرگ را به کوچک و کوچک را به بزرگ تبدیل کند.

```
void Dual_Inversion(char str[]) {
}
```

**نکته مهم: استفاده از هیچ کتابخانه ای مجاز نیست.

راهنمایی: می توانید از جدول اسکی زیر استفاده کنید.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
32	20	[SPACE]	64	40	@	96	60	`
33	21	!	65	41	A	97	61	a
34	22	"	66	42	B	98	62	b
35	23	#	67	43	C	99	63	c
36	24	\$	68	44	D	100	64	d
37	25	%	69	45	E	101	65	e
38	26	&	70	46	F	102	66	f
39	27	'	71	47	G	103	67	g
40	28	(	72	48	H	104	68	h
41	29	)	73	49	I	105	69	i
42	2A	*	74	4A	J	106	6A	j
43	2B	+	75	4B	K	107	6B	k
44	2C	,	76	4C	L	108	6C	l
45	2D	-	77	4D	M	109	6D	m
46	2E	.	78	4E	N	110	6E	n
47	2F	/	79	4F	O	111	6F	o
48	30	0	80	50	P	112	70	p
49	31	1	81	51	Q	113	71	q
50	32	2	82	52	R	114	72	r
51	33	3	83	53	S	115	73	s
52	34	4	84	54	T	116	74	t
53	35	5	85	55	U	117	75	u
54	36	6	86	56	V	118	76	v
55	37	7	87	57	W	119	77	w
56	38	8	88	58	X	120	78	x
57	39	9	89	59	Y	121	79	y
58	3A	:	90	5A	Z	122	7A	z
59	3B	;	91	5B	[	123	7B	{
60	3C	<	92	5C	\	124	7C	
61	3D	=	93	5D	]	125	7D	}
62	3E	>	94	5E	^	126	7E	~
63	3F	?	95	5F	_	127	7F	[DEL]

```
void Dual_Inversion(char str[]) {
    int len = 0, i = 0;

    while(str[i] != '\0'){
        len++;
        i++;
    }

    int len2 = (len % 2 == 0) ? len / 2 : (len / 2) + 1;
    char temp;

    for(int i = 0; i < len2; i++){
        if(97 <= (int)str[i] && (int)str[i] <= 122){
            temp = (char)((int)str[i] - 32);
        } else if(65 <= (int)str[i] && (int)str[i] <= 90){
            temp = (char)((int)str[i] + 32);
        } else {
            temp = str[i];
        }

        if(97 <= (int)str[len-1-i] && (int)str[len-1-i] <= 122){
            str[i] = (char)((int)str[len-1-i] - 32);
        } else if(65 <= (int)str[len-1-i] && (int)str[len-1-i] <= 90){
            str[i] = (char)((int)str[len-1-i] + 32);
        } else {
            str[i] = str[len-1-i];
        }

        str[len-1-i] = temp;
    }
}
```